

• 오라클 11일차

• VIEW 정리

- 단순 - 하나의 테이블에서만 데이터를 가져온다. 즉 조인문장이 없다
 - 함수는 데이터그룹을 포함하지 않음
 - 뷰를 통해서 DML 작업을 수행
- 복합 - 여러 테이블에서 데이터를 가져옴 즉 조인문장이 있는 경우
 - 함수나 데이터 그룹을 포함
 - 뷰를 통해서 DML작업을 수행할 수 없음
 - 복합뷰에서 DML작업을 수행하려면 PLSQL 언어를 이용해서 트리거 프로그램으로 구현
- DROP TABLE hr.emp PURGE;
- CREATE TABLE hr.emp
TABLESPACE users
- AS
- SELECT employee_id id, last_name name, salary sal
- FROM hr.employees
- WHERE department_id = 20;
- SELECT * FROM hr.emp;
- CREATE OR REPLACE VIEW hr.emp_view
- AS
- SELECT * FROM hr.emp;
- SELECT "ID","NAME","SAL" FROM hr.emp;
-
- SELECT * FROM user_objects WHERE object_name IN ('EMP','EMP_VIEW');
- SELECT * FROM user_views WHERE view_name = 'EMP_VIEW';
- SELECT text FROM user_views WHERE view_name = 'EMP_VIEW';
-
- desc hr.emp_view;
-
- INSERT INTO hr.emp_view(id,name,sal) values (300,'James',5000);
- SELECT * FROM hr.emp_view;
- SELECT * FROM hr.emp;
- COMMIT;
-
- INSERT INTO hr.emp_view(id,sal) values(400,6000);
- -- NOT NULL로 인해 name 값이 누락이 되어 오류 발생
-
- CREATE OR REPLACE VIEW hr.emp_view
- AS
- SELECT id,sal FROM hr.emp;

-
- SELECT * FROM hr.emp_view;
-
- INSERT INTO hr.emp_view(id,name,sal) values (400,'Oracle',6000);
- -- view가 보는 것은 id, sal 현재 두가지를 보고 있는 상황에서 Oracle < name값이 들어와 오류 발생
-
-
- -- insa 필드
- SELECT * FROM user_tab_privs;
-
- SELECT * FROM hr.emp_view;
-
- desc hr.emp_view;
-
- INSERT INTO hr.emp_view (id,sal) values(500,6000);
- -- 뷰를 통해서 볼 수 없는 컬럼들 중 not_null 제약조건이 있는 경우 INSERT 오류
-
- -- 단순뷰는 뷰를 통해서, 수정, 삭제 가
- UPDATE hr.emp_view
- SET sal = 5000
- WHERE id = 202;
-
- DELETE FROM hr.emp_view WHERE id = 201;
-
- SELECT * FROM hr.emp_view WHERE id = 201;
-
- COMMIT;
-
- hr 필드
-
- INSERT INTO hr.emp_view(id,sal) values(400,6000);
- -- view에서 선택되지 않은 컬럼 중에 not_null 제약조건이 있는 경우 오류
-
- -- 단순 뷰는 뷰를 통해서 수정 가능
- UPDATE hr.emp_view
- SET sal = 6000
- WHERE id = 300;
-
- SELECT * FROM hr.emp_view;
- SELECT * FROM hr.emp;
- -- 변화된 값을 같이 볼 수 있지만 커밋이 되지 않아 다른 세션에서는 이전 값을 봐야 됨
-
- COMMIT;

- DELETE FROM hr.emp_view WHERE id = 300;
- SELECT * FROM hr.emp_view;
- SELECT * FROM hr.emp;
- COMMIT;
- GRANT SELECT,INSERT, UPDATE, DELETE ON hr.emp_view TO insa;
- SELECT * FROM user_tab_privs;
- REVOKE ALL ON hr.emp_view FROM insa;
- SELECT * FROM user_tab_privs;
- CREATE OR REPLACE VIEW hr.emp_view
- AS
- SELECT * FROM hr.emp;
- SELECT * FROM hr.emp_view;
- INSERT INTO hr.emp_view(id,name,sal)
- SELECT employee_id, last_name, salary
- FROM hr.employees
- WHERE employee_id <> 202;
- COMMIT;
- SELECT * FROM hr.emp_view;
- SELECT * FROM hr.emp;
- -- 단순뷰지만 DML은 불어하도록 하는 방법 - WITH READ ONLY
- CREATE OR REPLACE VIEW hr.emp_view
- AS
- SELECT * FROM hr.emp
- WITH READ ONLY;
- SELECT * FROM hr.emp_view;
- INSERT INTO hr.emp_view(id,name,sal) values(300,'Oracle',5000);
- -- 단순뷰지만 WITH READ ONLY로 설정되어 있기 때문에 dml 작업 수행 불가
- DROP VIEW hr.emp_view;
- DROP TABLE hr.emp PURGE;
- CREATE TABLE hr.emp
- TABLESPACE users

- AS
- SELECT employee_id id, last_name name, salary sal, department_id dept_id
- FROM hr.employees;

- CREATE OR REPLACE VIEW hr.emp_20
- AS
- SELECT * FROM hr.emp WHERE dept_id = 20;

- SELECT * FROM hr.emp_20;

- INSERT INTO hr.emp_20(id,name,sal,dept_id) values(300,'Oracle',5000,30);

- SELECT * FROM hr.emp WHERE dept_id = 30;

- ROLLBACK;

- -- 뷰에 check 제약조건을 생성하는 방법 - WITH CHECK OPTION, view에
 생성되는 check 제약조건은 조건식은 where 절 조건식이 된다.
- CREATE OR REPLACE VIEW hr.emp_20
- AS
- SELECT * FROM hr.emp WHERE dept_id = 20
- WITH CHECK OPTION CONSTRAINT emp_20_ck;

- SELECT * FROM user_constraints;
- SELECT * FROM user_constraints WHERE table_name = 'EMP_20';

- INSERT INTO hr.emp_20(id,name,sal,dept_id) values(300,'Oracle',5000,30);
- -- 뷰에 check 제약조건이 만들어져 있기 때문에 check 제약 조건에 위반

- INSERT INTO hr.emp_20(id,name,sal,dept_id) values(300,'Oracle',5000,20);

- INSERT INTO hr.emp_20(id,name,sal,dept_id)
 values(400,'James',6000,NULL);
- -- not_null 제약조건으로 인한 오류

- ROLLBACK;

- UPDATE hr.emp_20
- SET sal = 7000
- WHERE id = 202;

- -- 뷰를 통해서 다른 부서 코드로 수정 불가 체크제약조건 위반
- UPDATE hr.emp_20
- SET dept_id = 10
- WHERE id = 201;

- -- 뷰를 통해서 삭제 가능 DML 허용
- DELETE FROM hr.emp_20 WHERE id = 201;

- ROLLBACK;
- -- SEQUENCE
 - -- 자동 일련번호를 생성하는 객체
 - -- CREATE SEQUENCE 시스템 권한
 - -- 시퀀스 생성
 - CREATE SEQUENCE hr.id_seq;
 - SELECT * FROM user_sequences WHERE sequence_name = 'ID_SEQ';
 - -- 시퀀스 삭제
 - DROP SEQUENCE hr.id_seq;
 - CREATE SEQUENCE hr.id_seq
 - START WITH 1 -- 기본값
 - MAXVALUE 10 -- 기본값은 10²⁷
 - INCREMENT BY 1 -- 기본값
 - NOCYCLE -- 기본값, CYCLE
 - NOCACHE ; -- 기본값은 CACHE 20
 - SELECT * FROM user_sequences WHERE sequence_name = 'ID_SEQ';
 - DROP TABLE hr.emp PURGE;
 - CREATE TABLE hr.emp(id number, name varchar2(30), day timestamp) TABLESPACE users;
 - desc hr.emp;
 - -- 의사(가상) 컬럼(pseudo column)은 실제 컬럼이 아니지만 컬럼처럼 사용되는 객체
 - -- 시퀀스 이름.nextval : 가상컬럼(의사컬럼(의사열)), 다음 사용가능한 번호 리턴
 - INSERT INTO hr.emp(id,name,day) values(id_seq.nextval, 'Oracle', localtimestamp);
 - SELECT * FROM hr.emp;
 - COMMIT;
 - SELECT id_seq.currval FROM dual;
 - -- nextval를 사용시에는 주의를 해야 된다 조회 중에 사용으로 인해 다음 값 생성
 - SELECT id_seq.nextval FROM dual;
 - -- 이로인해 ID값에 다음 값이 적용
 - INSERT INTO hr.emp(id,name,day) values(id_seq.nextval, 'Oracle', localtimestamp);

- SELECT * FROM hr.emp;
- SELECT id_seq.nextval FROM dual;
- SELECT id_seq.currval FROM dual;
- SELECT * FROM user_sequences WHERE sequence_name = 'ID_SEQ';
- -- 이를 해결하기 위해 nextval을 사용하지 않고 currval을 사용하여 다음 값 출력
- INSERT INTO hr.emp(id,name,day) values(id_seq.currval, 'Liam', localtimestamp);
- SELECT * FROM hr.emp;
- COMMIT;
- SELECT * FROM user_sequences WHERE sequence_name = 'ID_SEQ';
- INSERT INTO hr.emp(id,name,day) values (id_seq.nextval,'Olivia',localtimestamp);
- INSERT INTO hr.emp(id,name,day) values (id_seq.nextval,'Leo',localtimestamp);
- INSERT INTO hr.emp(id,name,day) values (id_seq.nextval,'Noah',localtimestamp);
- COMMIT;
- SELECT * FROM hr.emp;
- INSERT INTO hr.emp(id,name,day) values (id_seq.nextval,'Asher',localtimestamp);
- INSERT INTO hr.emp(id,name,day) values (id_seq.nextval,'Luna',localtimestamp);
- SELECT * FROM hr.emp;
- ROLLBACK;
- SELECT * FROM user_sequences WHERE sequence_name = 'ID_SEQ';
- -- 시퀀스 객체는 한번 적용된 번호는 다시 사용할 수 없음
- INSERT INTO hr.emp(id,name,day) values (id_seq.nextval,'Asher',localtimestamp);
- -- 시퀀스 MAXVALUE값까지 사용한 경우 다음 nextval 값이 생성되지 않기 때문에 오류
- INSERT INTO hr.emp(id,name,day) values (id_seq.nextval,'Luna',localtimestamp);
- -- 시퀀스 수정
- -- START WITH 절을 제외하고 다른 옵션들을 수정가능

- ALTER SEQUENCE id_seq
- MAXVALUE 100
- INCREMENT BY 2
- CACHE 20;

- SELECT * FROM user_sequences WHERE sequence_name = 'ID_SEQ';

- PL/SQL (procedure Language Structured Query Language)
 - SQL을 확장한 절차적 언어
 - Procedure(프로시저) 생성자를 제공
 - 변수, 상수, 데이터유형(레코드, 배열)
 - if 조건문, Loop 반복문
 - 예외사항 처리

 - 이점
 - 모듈식 프로그램 개발
 - 블록내의 관련 명령문을 논리적으로 그룹화
 - 즉 프로그램을 만들어 재구성(재조립)가능
 - 블록을 더 큰 블록 내부에 중첩하여 강력한 프로그램 작성
 - 응용프로그램을 더 작은 모듈로 분리
 - 코드를 쉽게 유지 관리 가능
 - 디버그가 용의
 - 오라클 도구와 통합(ERP, HR, SCM, CRM)
 - 이식성 : 운영체제나 플랫폼에 상관없이 오라클 서버가 실행되는 모든 환경에서 실행이 가능

 - 익명(ANONYMOUS) 블록구조
 - 객체 프로그램이 아니다 (데이터베이스에 저장되지 않음)
 - DECLARE (선택)
 - 선언
 - 변수, 상수, 명시적커서, 사용자 정의 예외사항
 - BEGIN (필수)
 - 실행부분
 - SQL(SELECT, INSERT, UPDATE, DELETE, MERGE, COMMIT, ROLLBACK, SAVEPOINT)
 - 반복문, 조건문
 - EXCEPTION (선택)
 - 예외사항 (실행부분에서 발생한 오류에 대한 처리 #
 - END; (필수)
 - /
 - dbms_output.put_line : 출력하는 프로그램
 - BEGIN
 - dbms_output.put_line('문자열을 출력');
 - dbms_output.put_line('문자열을 출력.1// 한줄씩 출력');
 - dbms_output.put_line('문자열을 출력.2');

- END;
- /
- 환경설정을 해줘야 된다. **set serveroutput on**
- 출력 값을 보기 위해서는 보기에 **DBMS** 보기를 켜야 된다.
- 변수
 - 데이터를 임시로 저장하는 메모리 영역
 - 문자로 시작
 - 문자, 숫자, 특수문자(underscore, hash, dollar)포함
 - 변수이름의 길이는 30자 이하만 가능
 - 예약어는 사용 불가
 - SELECT keyword FROM v\$reserved_words WHERE reserved = 'Y';
 - 변수 선언시에 NOT NULL, CONSTANT(상수)로 지정된 변수에는 꼭 초기값 할당
 - 변수에 값을 할당하는 연산자(:=, default)
 - 변수는 변하는 값을 계속 입력 가능
 - 상수는 한번 받은 값만 계속 사용
 - 한행에 하나씩 변수, 상수를 선언
 - Ex) v_id, v_no number; – 오류
 - PL/SQL)
 - v_id number;
 - v_no number;
 - DECLARE
 - v_name varchar2(30);
 - BEGIN
 - dbms_output.put_line(' My name is'||v_name);
 - v_name := 'ORACLE';
 - dbms_output.put_line('My name is' || v_name);
 - END;
 - /
 - BEGIN
 - dbms_output.put_line('Today"s : ' || to_char(sysdate,'YYYY-MM-DD'));
 - dbms_output.put_line('Tomorrow"s: ' || to_char(sysdate+1,'YYYY-MM-DD'));
 - END;
 - /
- BEGIN
 - dbms_output.put_line(q'[Today"s :]' || to_char(sysdate,'YYYY-MM-DD'));
 - dbms_output.put_line(q'[Tomorrow"s:]' || to_char(sysdate+1,'YYYY-MM-DD'));
- END;
- /

- SELECT keyword FROM v\$reserved_words WHERE reserved = 'Y';

- DECLARE
- v_job VARCHAR2(50) := 'Oracle Enginner';
- BEGIN
- dbms_output.put_line('I work as a'|| v_job);
- v_job := 'Oracle DBA';
- dbms_output.put_line('I work as a '|| v_job);
- END;
- /

- DECLARE
- v_a number(5);
- v_b number(3):= 100;
- v_c varchar2(10) NOT NULL := 'oracle';
- v_d CONSTANT date DEFAULT sysdate;
- v_e CONSTANT number(3) := 20;
- BEGIN
- v_a := 200;
- dbms_output.put_line(v_a);
- dbms_output.put_line(v_b);
- dbms_output.put_line(v_c);
- dbms_output.put_line(v_d);
- dbms_output.put_line(v_e);
- END;
- /

- DECLARE
- – 지역변수로 선언된 블록 프로그램에서만 수행
- v_sal number := 1000;
- v_comm number := 0.1;
- v_total number;
- BEGIN
- v_total := v_sal * 12 + v_sal * 12 * v_comm;
- dbms_output.put_line(v_total);
- END;
- /

- -- 외부에서 사용하고 싶지만 식별자가 없어서 에러 발생
- SELECT * FROM hr.employees WHERE salary >= v_total;
- 전역변수 : 프로그램 바깥쪽에서도 사용할 수 있는 변수

- 바인드변수(bind variable) - 호스트 변수(host variable)
 - 호스트 환경에서 생성한 변수(프로그램 바깥쪽에서 선언)
 - variable(var) 키워드 사용해서 선언
 - 바인드 변수는 SQL문, PL/SQL(익명블록)에서 사용

- 바인드 변수를 사용할 때는 콜론(:) 키워드를 바인드 변수 이름 앞에 접두어로 사용
- global variable(전역변수) 처럼 사용하고 싶을 때 사용
 - 프로그램 어디서든 사용하는 변수
 - 즉 세션이 열린 동안 어디서든 사용 가능
 - var b_id number
 - execute :b_id := 100
 - print b_id
 - SELECT last_name, salary FROM hr.employees WHERE employee_id = : b_id;
 - 바인드 변수 선언
 - var b_total number
 - print b_total
 - DECLARE
 - v_sal number :=1000;
 - v_comm number := 0.1;
 - BEGIN
 - :b_total := v_sal * 12 + v_sal * 12 * v_comm;
 - dbms_output.put_line(:b_total);
 - END;
 - /
 - print b_total
 - ALTER SYSTEM FLUSH SHARED_POOL;
 - SELECT *
 - FROM v\$sql
 - WHERE sql_text LIKE '%hr.employees%'
 - AND sql_text NOT LIKE '%v\$sql%';
 - SELECT * FROM table
 - (dbms_xplan.display_cursor('7s0rxjt3f50a9'));
 - -- 바인드변수 처리
 - SELECT * FROM hr.employees WHERE employee_id = :b_id;
 -